

Table des matières

Labo 05 — D'un dossier de scripts à un pipeline	1
Étape 1 — Ranger les scripts dans un paquet.....	2
Étape 2 — Sortir la configuration et le mot de passe du code	3
Étape 3 — Journaliser dans un fichier et dans la table des exécutions	6
Étape 4 — Réessayer les appels à l'API qui échouent en passant	10
Étape 5 — Les étapes, et la porte d'entrée	12
Étape 6 — Écrire trois tests.....	19
Étape 7 — Mettre le pipeline dans une image, et la lancer.....	23
Étape 8 — Versionner le code, laisser dehors l'image et les données	25
Récapitulatif	25
Dépannage	26
Annexe — Tout le code, d'un seul tenant.....	27

Labo 05 — D'un dossier de scripts à un pipeline

Labo · Module 5/12 — Formation Data Engineering

Ce que vous allez faire. Reprendre les scripts des Cours 02 à 04 et en faire un paquet Python avec une seule porte d'entrée, `python -m pipeline`. Sortir la configuration du code et le mot de passe du dépôt. Brancher une journalisation qui se lit et une table d'exécutions qui se compte. Faire des reprises de l'API un outil commun. Écrire trois tests qui tournent en moins d'une seconde. Construire une image Docker du pipeline et la lancer sur le projet monté. Objectif de fin : trois tests verts, `python -m pipeline tout` qui charge 433 582 lignes de bronze et 513 857 lignes de RAW puis ne recharge rien au second passage, et la même commande qui tourne dans un conteneur.

Comptez une séance de deux heures et demie à trois heures, la frappe comprise : c'est le labo le plus dense de la formation. Qui le fait en deux fois s'arrête après l'étape 5, quand le pipeline tourne.

Vous partez du dépôt 01_de tel que le Labo 04 l'a laissé : `sql/` avec les trois scripts SQL, `ingestion/` avec les quatre scripts Python, `.env.exemple` et `.gitignore`. La couche RAW est chargée, 22 tables et 513 857 lignes.

Deux terminaux servent dans ce labo. L'Anaconda Prompt, environnement `formations` actif, pour Python, Git et DuckDB ; Ubuntu dans WSL 2 pour Docker, à partir de l'étape 7. Chaque étape dit dans lequel on se trouve.

Un outil de plus sert à partir d'ici, pytest, arrivé avec les paquets de la formation à l'étape 2 du guide d'installation du parcours. Vérifiez-le dans l'Anaconda Prompt :

```
pytest --version
```

Il répond son numéro de version. Sinon, `pip install -r 01_Data_Engineering\requirements.txt`, depuis Data_Domain.

La liste de contrôle du cours :

- ☐ Le paquet pipeline créé et les quatre scripts des Cours 03 et 04 déplacés par `git mv :dir pipeline` montre `__init__.py`, `raw.py`, `meteo.py`, `erp.py` et `corpus.py`
- ☐ La configuration hors du code et le mot de passe hors du dépôt : le port lu rend 5433, puis 5434 sous `PIPELINE_ERP_PORT`, et reste un entier
- ☐ Le journal branché : un fichier par jour dans `plateforme/journaux/`, et une ligne par passage dans `pipeline.executions`
- ☐ Les reprises mises en commun : `meteo.py` n'a plus sa boucle d'essais, `reprise.py` la porte pour tout le pipeline
- ☐ Les cinq modules d'étape et la porte d'entrée : `findstr /R /C:"^def executer" pipeline\*.py` rend cinq lignes, et `python -m pipeline` tout écrit 433 582 lignes de bronze, 513 857 en RAW sur 22 tables et 35 fichiers de corpus ; le second passage n'en recharge aucune
- ☐ Trois tests verts : `pytest` rend 3 `passed` en moins d'une seconde
- ☐ L'image `boreal-pipeline` construite et lancée sur le projet monté : les mêmes comptes qu'à l'étape 5, l'étape `raw` en 6,5 s contre 0,1 s sous Windows
- ☐ Le code versionné et `.env` absent du `git status` d'avant le commit

Étape 1 — Ranger les scripts dans un paquet

Théorie : Theorie_05.md, section 1.

Dans l'Anaconda Prompt, depuis `Data_Domain\Projet_Boreal\01_de`.

Un paquet Python est un dossier qui porte un fichier `__init__.py`. Créons-en deux, celui du pipeline et celui des tests, et déplaçons les scripts des Cours 03 et 04 vers leur place définitive.

```
mkdir pipeline
mkdir tests
git mv ingestion\charger_raw.py pipeline\raw.py
git mv ingestion\extraire_meteo_api.py pipeline\meteo.py
git mv ingestion\extraire_erp_odbc.py pipeline\erp.py
```

```
git mv ingestion\deposer_corpus.py pipeline\corpus.py
rmdir ingestion
```

Ces quatre fichiers ne marchent plus en l'état : ils se lançaient seuls, ouvraient l'entrepôt chacun de leur côté et écrivaient à l'écran. L'étape 4 explique ce qui change dans la météo, et l'étape 5 les reprend tous les quatre, l'annexe sous les yeux. Tant que cette reprise n'est pas faite, python -m pipeline meurt à l'import. Le paquet lui-même se déclare maintenant. Créez le fichier :

```
notepad pipeline\__init__.py

# -*- coding: utf-8 -*-
"""pipeline : le code du pipeline de Véhicules Boréal inc.

Un paquet, une porte d'entrée : python -m pipeline <étape>. Les modules de fond (config, journal,
reprise, entrepot) servent toutes les étapes ; Les modules d'étape (bronze, raw, corpus, erp, meteo)
ne connaissent que leur source et exposent chacun une fonction executer(con).
"""

__version__ = "0.3.0"
```

Piège — le fichier enregistré en .txt. Le Bloc-notes ajoute .txt quand on enregistre sans choisir. Dans la boîte d'enregistrement, mettez « Type » à « Tous les fichiers » et tapez le nom exactement, __init__.py. Si dir pipeline montre __init__.py.txt, renommez : ren pipeline__init__.py.txt __init__.py.

Étape 2 — Sortir la configuration et le mot de passe du code

Théorie : Theorie_05.md, sections 2 et 3.

Le port de la base, la station de météo, le nombre d'essais : autant de valeurs qui changeraient sur un autre poste. Elles sortent du code. Le mot de passe, lui, sort du dépôt, ce qui n'est pas la même chose (théorie, sections 2 et 3).

```
notepad config.toml

# config.toml : tout ce que le pipeline de Boréal sait de son environnement.
#
# La configuration sort du code, mais pas les secrets : aucun mot de passe ici, ce fichier va dans
# Git. Une valeur se surcharge sans toucher au fichier, par une variable d'environnement
# PIPELINE_<SECTION>_<CLE> : PIPELINE_ERP_PORT=5434 gagne sur port = 5433 ci-dessous.
#
# Les chemins sont relatifs à Projet_Boreal, le dossier du projet, que le pipeline retrouve seul.
```

```
[chemins]
sources = "sources"
lac = "plateforme/lac"
entrepot = "plateforme/entrepot/boreal.duckdb"
```

```

journaux = "plateforme/journaux"

[journal]
niveau = "INFO"

[reprises]
essais = 4
attente = 2.0

[erp]
serveur = "localhost"
port = 5433
base = "erp_boreal"
utilisateur = "postgres"
paquet = 10000
tables = ["t005u", "kna1", "mara", "makt", "vbak", "vbap", "cdhdr", "cdpos", "zserie"]

[meteo]
url = "https://api.weather.gc.ca/collections/climate-daily/items"
station = "7060400"
debut = "2025-10-01"
fin = "2025-12-31"
page = 50

```

Le module qui le lit. lru_cache de la bibliothèque standard garde le résultat du premier appel et rend le même ensuite : la configuration se lit une fois, pas à chaque étape.

```

notepad pipeline\config.py

# -*- coding: utf-8 -*-
"""config.py : La configuration et les secrets du pipeline, lus une fois pour toutes."""
import os
import tomllib
from functools import lru_cache
from pathlib import Path

# Le dépôt est le dossier qui porte le paquet, et il ne bouge jamais.
DEPOT = Path(__file__).resolve().parents[1]

def projet() -> Path:
    """Le dossier Projet_Boreal : le parent du dépôt, sauf si PIPELINE_PROJET en désigne un autre."""

```

La variable déplace le projet sans déplacer le code : Le conteneur s'en sert pour désigner le dossier monté, et les tests pour travailler dans un dossier temporaire.

```
return Path(os.environ.get("PIPELINE_PROJET")) or DEPOT.parent)
```

À la suite, dans le même fichier, la lecture du TOML. Une valeur se remplace sans qu'on touche au fichier, et c'est le type de la valeur lue qui commande la conversion : un port reste un entier, une liste de tables reste une liste. `lru_cache` fait lire le fichier une seule fois par exécution, pour que deux étapes d'un même passage ne travaillent jamais sur deux configurations différentes. Laissez deux lignes vides avant ce qui suit : c'est la respiration que Python met entre deux fonctions de module, et tous les fichiers coupés en morceaux dans ce labo se recollent ainsi.

```
def _surcharger(valeurs: dict) -> dict:
    """Remplace une valeur par PIPELINE_<SECTION>_<CLE> quand l'environnement en porte une.

    Le type de la valeur du fichier commande la conversion : un port reste un entier, une liste se
    coupe sur les virgules.
    """
    for section, contenu in valeurs.items():
        if not isinstance(contenu, dict):
            continue
        for cle, default in contenu.items():
            brut = os.environ.get(f"PIPELINE_{section.upper()}_{cle.upper()}")
            if brut is None:
                continue
            if isinstance(default, bool):
                contenu[cle] = brut.strip().lower() in ("1", "true", "oui")
            elif isinstance(default, list):
                contenu[cle] = [morceau.strip() for morceau in brut.split(",") if morceau.strip()]
            elif isinstance(default, (int, float)):
                contenu[cle] = type(default)(brut)
            else:
                contenu[cle] = brut
    return valeurs
```

```
@lru_cache(maxsize=1)
def config() -> dict:
    """La configuration du pipeline, lue une seule fois par exécution."""
    fichier = DEPOT / "config.toml"
    if not fichier.exists():
        raise FileNotFoundError(f"{fichier} est introuvable : le pipeline ne sait pas où il tourne.")
    with open(fichier, "rb") as source:
        return _surcharger(tomllib.load(source))
```

Et pour finir, les deux fonctions que le reste du pipeline appelle vraiment. Un chemin se demande par son nom, jamais en dur ; un secret se demande à l'environnement d'abord, au `.env` ensuite, ce qui laisse un conteneur travailler sans fichier de secrets.

```
def chemin(nom: str) -> Path:
    """Un chemin de la section [chemins], rendu absolu depuis Projet_Boreal."""
    return projet() / config()["chemins"][nom]

def secret(cle: str) -> str:
    """Une valeur de 01_de/.env, un fichier de lignes CLE=valeur que Git ignore.

    L'environnement gagne sur le fichier : un conteneur reçoit ses secrets par lui, sans .env.
    """
    if cle in os.environ:
        return os.environ[cle]
    env = DEPOT / ".env"
    if env.exists():
        for ligne in env.read_text(encoding="utf-8").splitlines():
            if ligne.lstrip().startswith("#"):
                continue
            nom, _, valeur = ligne.partition("=")
            if nom.strip() == cle:
                return valeur.strip()
    raise KeyError(f"{cle} n'est ni dans l'environnement ni dans {env} : "
                  f"copiez .env.exemple en .env et remplissez-le.")
```

Vérifiez que la configuration se lit, et que l'environnement gagne bien sur le fichier :

```
python -c "from pipeline.config import config; print(config()['erp']['port'])"
set PIPELINE_ERP_PORT=5434
python -c "from pipeline.config import config; print(config()['erp']['port'])"
set PIPELINE_ERP_PORT=
```

Attendu : 5433, puis 5434, et le type reste un entier. La troisième commande remet l'environnement en place ; si vous l'oubliez, le pipeline cherchera la base sur le mauvais port jusqu'à la fermeture de ce terminal.

À retenir — deux raisons, deux fichiers. `config.toml` est versionné : son historique raconte les décisions. `.env` ne l'est jamais : Git n'oublie rien, et un mot de passe retiré au commit suivant reste lisible dans le précédent.

Étape 3 — Journaliser dans un fichier et dans la table des exécutions

Théorie : Theorie_05.md, section 4.

contextmanager fabrique un bloc with : ce qui est écrit avant le yield se fait à l'entrée, ce qui suit se fait à la sortie, même quand le bloc lève une erreur. C'est ce qui garantit qu'une exécution finit toujours par écrire sa ligne de bilan.

Deux traces pour deux lecteurs : un texte daté pour celui qui cherche pourquoi, une table pour celui qui compte combien (théorie, section 4).

```
notepad pipeline\journal.py

# -*- coding: utf-8 -*-
"""journal.py : ce que le pipeline raconte pendant qu'il tourne, et ce qu'il en garde."""
import logging
import sys
import time
import uuid
from contextlib import contextmanager
from datetime import datetime, timezone

from . import config as conf

LOG = logging.getLogger("pipeline")

def demarrer() -> logging.Logger:
    """Branche la journalisation : la console, et un fichier par jour."""
    reglages = conf.config()["journal"]
    dossier = conf.chemin("journaux")
    dossier.mkdir(parents=True, exist_ok=True)
    fichier = dossier / f"pipeline_{datetime.now().strftime('%Y-%m-%d')}.log"

    forme = logging.Formatter("%(asctime)s %(levelname)-7s %(name)s : %(message)s",
                              datefmt="%Y-%m-%d %H:%M:%S")

    LOG.handlers.clear()
    LOG.setLevel(reglages["niveau"])
    for sortie in (logging.StreamHandler(sys.stdout),
                  logging.FileHandler(fichier, encoding="utf-8")):
        sortie.setFormatter(forme)
        LOG.addHandler(sortie)
    LOG.debug("journal ouvert dans %s", fichier)
    return LOG
```

À la suite, la seconde trace, celle qui se compte : la table des exécutions, et le petit objet qu'une étape reçoit pour dire ce qu'elle a écrit.

```
def creer_table(con) -> None:
    """La table des exécutions, dans le schéma pipeline, à côté du journal des chargements."""
```

```

con.execute("CREATE SCHEMA IF NOT EXISTS pipeline")
con.execute("""
    CREATE TABLE IF NOT EXISTS pipeline.executions (
        execution_id VARCHAR,      -- huit caractères, pour retrouver l'exécution dans le .log
        etape          VARCHAR,
        debut          TIMESTAMPTZ,
        fin            TIMESTAMPTZ,
        duree_s        DOUBLE,
        etat           VARCHAR,      -- en cours, réussi, échoué
        lignes         BIGINT,      -- ce que l'étape a écrit
        message        VARCHAR)""")

```

```
class Trace:
```

```
    """Ce qu'une étape a le droit de dire d'elle-même pendant qu'elle tourne."""
```

```

def __init__(self, execution_id: str, etape: str):
    self.execution_id = execution_id
    self.etape = etape
    self.lignes = None
    self.message = None

```

Vient le cœur du module. `etape` encadre le travail d'une étape : elle écrit sa ligne avant de la lancer, la referme à la fin, et la referme aussi quand l'étape tombe. Lisez le bloc `except` en entier : ce qu'il fait dans l'ordre est ce qui sépare une trace fiable d'une trace qui ment.

```
@contextmanager
```

```
def etape(con, nom: str):
```

```
    """Encadre une étape : une ligne dans pipeline.executions, et le temps qu'elle a pris.
```

```

    La ligne s'écrit avant le travail, à l'état « en cours » : si le processus est tué en route, une
    exécution qui n'a jamais fini se voit.
    """

```

```

    creer_table(con)
    trace = Trace(uuid.uuid4().hex[:8], nom)
    debut = datetime.now(timezone.utc)
    chrono = time.perf_counter()
    con.execute("INSERT INTO pipeline.executions VALUES (?, ?, ?, NULL, NULL, 'en cours', NULL, NULL)",
                [trace.execution_id, nom, debut])
    LOG.info("étape %s (%s) : début", nom, trace.execution_id)
    try:
        yield trace
    except BaseException as erreur:
        # La trace doit survivre à ce qui l'a interrompue. Deux précautions, dans cet ordre.

```

```

# D'abord annuler la transaction que l'étape aurait laissée ouverte : sans cela, l'UPDATE
# part avec elle, et la ligne reste à « en cours » en attendant. Ensuite écrire la trace sous
# son propre try : si elle échoue à son tour, c'est la cause réelle qui doit remonter, pas
# une erreur de journalisation.
try:
    con.execute("ROLLBACK")
except Exception:
    pass # aucune transaction ouverte : rien à annuler
try:
    _fermer(con, trace, debut, chrono, "échoué", f"{type(erreur).__name__} : {erreur}")
except Exception as souci:
    LOG.error("étape %s (%s) : trace non écrite (%s)", nom, trace.execution_id, souci)
    LOG.error("étape %s (%s) : échec, %s", nom, trace.execution_id, erreur)
raise
_fermer(con, trace, debut, chrono, "réussi", trace.message)
LOG.info("étape %s (%s) : fin, %s en %.1f s", nom, trace.execution_id,
        "aucune ligne" if trace.lignes is None else f"{trace.lignes:,} lignes".replace(",", " "),
        time.perf_counter() - chrono)

```

```

def _fermer(con, trace: Trace, debut, chrono: float, etat: str, message) -> None:
    con.execute("""
        UPDATE pipeline.executions
        SET fin = ?, duree_s = ?, etat = ?, lignes = ?, message = ?
        WHERE execution_id = ? AND debut = ?""",
        [datetime.now(timezone.utc), time.perf_counter() - chrono, etat,
         trace.lignes, message, trace.execution_id, debut])

```

L'entrepôt n'est plus ouvert par chaque script, mais une seule fois, par le pipeline, et passé aux étapes. C'est la décision D67 du parcours, celle d'un seul processus à la fois sur le fichier.

```
notepad pipeline\entrepot.py
```

```

# -*- coding: utf-8 -*-
"""entrepot.py : La seule porte vers boreal.duckdb, et le journal des chargements."""
import duckdb

from .config import chemin
from .journal import LOG

```

```

class EntrepotOccupe(RuntimeError):
    """Un autre processus tient déjà le fichier de l'entrepôt."""

```

```

def ouvrir(lecture_seule: bool = False):
    """Une connexion à l'entrepôt, ses schémas et son journal des chargements en place."""
    fichier = chemin("entrepot")
    fichier.parent.mkdir(parents=True, exist_ok=True)
    try:
        con = duckdb.connect(str(fichier), read_only=lecture_seule)
    except duckdb.IOException as erreur:
        raise EntrepotOccupe(
            "L'entrepôt est occupé : fermez la ligne de commande DuckDB, puis relancez. "
            f"({erreur})" from erreur
        )
    LOG.debug("entrepôt ouvert : %s", fichier)
    if not lecture_seule:
        con.execute("CREATE SCHEMA IF NOT EXISTS raw")
        con.execute("CREATE SCHEMA IF NOT EXISTS pipeline")
        con.execute("""
            CREATE TABLE IF NOT EXISTS pipeline.chargements (
                fichier    VARCHAR,      -- le chemin du fichier, depuis Projet_Boreal
                empreinte  VARCHAR,      -- SHA-256 du contenu
                table_raw  VARCHAR,
                lot        VARCHAR,
                lignes     BIGINT,
                charge_le  TIMESTAMPTZ)""")
    return con

def deja_charge(con, table: str) -> dict:
    """Ce que le journal sait de cette table : à chaque fichier, l'empreinte de sa dernière version."""
    return dict(con.execute("""
        SELECT fichier, empreinte FROM pipeline.chargements WHERE table_raw = ?
        QUALIFY ROW_NUMBER() OVER (PARTITION BY fichier ORDER BY charge_le DESC) = 1""",
        [table])).fetchall())

```

Étape 4 — Réessayer les appels à l'API qui échouent en passant

Théorie : Theorie_05.md, section 5.

Le script de l'API du Cours 04 portait sa propre boucle d'essais. Elle sert maintenant à toute source qui peut faiblir : le courtier MQTT du Cours 09, une tâche de DAG au Cours 06.

notepad pipeline\reprise.py

```

# -*- coding: utf-8 -*-
"""reprise.py : réessayer ce qui est passager, abandonner ce qui ne l'est pas."""
import time
from typing import Callable

from .config import config
from .journal import LOG

class Abandon(RuntimeError):
    """Tous les essais ont été tentés, sans succès."""

def avec_reprises(action: Callable, passagere: Callable[[Exception], bool], quoi: str,
                  essais: int = None, attente: float = None, dormir=time.sleep):
    """Lance action(), en la réessayant tant que passagere() reconnaît l'erreur.

    quoi nomme ce qu'on tentait, pour le journal. dormir s'échange dans les tests : la reprise s'y
    vérifie sans attendre quatorze secondes.
    """

    reglages = config()["reprises"]
    essais = reglages["essais"] if essais is None else essais
    attente = reglages["attente"] if attente is None else attente

    for essai in range(1, essais + 1):
        try:
            return action()
        except Exception as erreur:
            if not passagere(erreur):
                raise # demande fausse : la répéter ne sert à rien
            if essai == essais:
                raise Abandon(f"{quoi} : abandon après {essais} essais ({erreur})") from erreur
            delai = attente * 2 ** (essai - 1)
            LOG.warning("%s : essai %d raté (%s), nouvel essai dans %.0f s", quoi, essai, erreur, delai)
            dormir(delai)

```

Reprenez maintenant pipeline\meteo.py, qui s'appelait extraire_meteo_api.py. Trois changements, et rien d'autre : sa boucle d'essais part dans reprise.py, ses constantes viennent de la configuration, ses print deviennent des lignes de journal. Le fichier complet est dans l'annexe « Tout le code, d'un seul tenant » ; voici ce qui remplace l'ancienne fonction lire_page :

```

def passagere(erreur: Exception) -> bool:
    """Un 429 ou un 5xx passeront peut-être ; un 4xx dit que la demande est fausse."""

```

```

if isinstance(erreur, urllib.error.HTTPError):
    return erreur.code == 429 or erreur.code >= 500
return isinstance(erreur, urllib.error.URLError)    # coupure, DNS, délai dépassé

```

```

def lire_page(url: str) -> dict:
    """Une page de l'API, réessayée tant que l'erreur est passagère."""
    def demander():
        with urllib.request.urlopen(url, timeout=60) as reponse:
            return json.load(reponse)
    return avec_reprises(demander, passagere, "une page de l'API météo")

```

Le module ne connaît plus le nombre d'essais ni l'attente : il ne garde que ce qui lui est propre, savoir quelles erreurs HTTP valent un second essai.

À retenir — ce qui se répète dans deux étapes n'appartient à aucune des deux. Une reprise, une journalisation, une lecture de configuration sont l'affaire du pipeline. L'étape ne garde que la connaissance de sa source.

Étape 5 — Les étapes, et la porte d'entrée

Théorie : Theorie_05.md, section 7.

Dans l'Anaconda Prompt, depuis 01_de. Cinq modules d'étape, et la commande qui les appelle.

Le premier est nouveau : il lance les scripts SQL du Cours 02, qui jusqu'ici se tapaient un par un dans la ligne de commande DuckDB.

```

notepad pipeline\bronze.py

# -*- coding: utf-8 -*-
"""bronze.py : Les scripts SQL du Cours 02, lancés par Le pipeline."""
from pathlib import Path

from .config import DEPOT, chemin
from .journal import LOG

# Le script, et Le dossier parent que son COPY partitionné attend.
SCRIPTS = [
    ("historien", "bronze_historien.sql", "bronze/usine"),
    ("télémetrie", "bronze_telemetrie.sql", "bronze/usine"),
]

class EcartDeConversion(RuntimeError):
    """La source et Le bronze ne comptent pas Le même nombre de lignes."""

```

```

def espacer(nombre: int) -> str:
    """275436 devient « 275 436 », comme partout ailleurs dans la formation."""
    return f"{nombre:,}".replace(",", " ")

def convertir(con, script: Path) -> tuple:
    """Exécute un script de conversion et rend son contrôle final : (lignes source, lignes bronze).

    Le script porte sa requête de contrôle en dernière instruction : c'est son résultat qu'on lit.
    """
    resultat = con.execute(script.read_text(encoding="utf-8")).fetchone()
    if resultat is None or len(resultat) != 2:
        raise EcartDeConversion(f"{script.name} ne finit pas par un contrôle à deux colonnes.")
    return int(resultat[0]), int(resultat[1])

def executer(con) -> int:
    """Refait le bronze de l'historien et de la télémétrie. Rend le total des lignes écrites."""
    total = 0
    for nom, fichier, parent in SCRIPTS:
        (chemin("lac") / parent).mkdir(parents=True, exist_ok=True)
        source, bronze = convertir(con, DEPOT / "sql" / fichier)
        if source != bronze:
            # Les nombres se mettent en forme un par un : appliquer le remplacement à la phrase
            # entière emporterait aussi sa virgule de ponctuation.
            raise EcartDeConversion(
                f"{nom} : {espacer(source)} lignes dans la source, {espacer(bronze)} dans le "
                "bronze. La conversion a perdu ou ajouté des lignes.")
        LOG.info(" %-12s %10s lignes, source et bronze d'accord", nom, espacer(bronze))
        total += bronze
    return total

```

Le SQL du Cours 02 ne bouge pas : il reste dans ses fichiers, lançable seul par duckdb -f quand on met les mains dedans. Python l'appelle et lit son contrôle. Ce qui change, c'est que le contrôle devient une condition : un écart arrête le pipeline au lieu de défiler à l'écran.

Les quatre autres modules sont les scripts des Cours 03 et 04, déplacés à l'étape 1. Chacun subit les mêmes retouches, qui tiennent en quatre points :

Ce qui part	Ce qui le remplace
def main(), le bloc if __name__ == "__main__", os.chdir	def executer(con), appelée par la porte d'entrée, qui se place dans Projet_Boreal
L'ouverture de l'entrepôt dans le script	la connexion reçue en argument : un seul processus, une seule connexion
Les print	LOG.info et LOG.debug
Les constantes en tête de fichier	config(), chemin() et secret()

Chaque executer rend un entier, ce que l'étape a écrit : c'est ce que la porte d'entrée inscrit dans la colonne lignes de pipeline.executions. Le contenu complet des quatre fichiers est dans l'annexe « Tout le code, d'un seul tenant ». Deux retouches méritent d'être vues ici.

Dans pipeline\raw.py, la fin de executer :

```
total_fichiers += fichiers
total_lignes += lignes
LOG.info(" RAW : %s fichier(s) chargé(s) à ce passage", f"{total_fichiers:,}".replace(",", " "))
return total_lignes
```

Le compte des fichiers va au journal de texte, celui des lignes est rendu à la porte d'entrée. Une étape qui rendrait un couple ferait échouer l'écriture dans la table, sur une erreur de conversion de type que rien ne relierait à sa cause.

Dans pipeline\erp.py, la première ligne de executer :

```
def executer(con=None) -> int:
    """Extrait Les neuf tables de la base. Rend Le total des lignes lues."""
    import pyodbc # dépendance de cette étape seulement
```

Un import à l'intérieur d'une fonction est inhabituel, et c'est voulu : pyodbc et le pilote psqLODBC ne servent qu'à cette extraction. Importé en tête de module, il empêcherait tout le pipeline de démarrer sur un poste qui ne l'a pas. Le chemin que rejoue le kit de départ d'Analytics Engineering n'importe que DuckDB et la bibliothèque standard.

Ces quatre-là, vous les avez déjà : ce sont vos fichiers des Cours 03 et 04, déplacés à l'étape 1. Reprenez-les un par un et appliquez-leur les quatre retouches du tableau. L'annexe « Tout le code, d'un seul tenant » donne le résultat, module par module, meteo.py comprise, dont l'étape 4 ne montrait qu'un extrait : travaillez avec elle sous les yeux, et recopiez-en ce que vous ne voulez pas réécrire. Ce qui compte est que le code laissé dans pipeline\ soit celui que vous avez lu.

Un seul fichier se copie, parce qu'il n'appartient pas à ce cours. Depuis 01_de :

```
copy /Y ..\..\01_Data_Engineering\Cours\Solution\01_de\pipeline\fenetre.py pipeline\
```

fenetre.py arrive au Cours 06 et donne au pipeline la notion de tranche de temps. Le Solution/ porte l'état final du dépôt, pas un instantané par cours, si bien que raw.py l'importe déjà, et que les quatre executer de l'annexe portent un paramètre fenetre et un paramètre livraisons dont ce cours-ci ne se sert pas. Sans borne, la fenêtre contient tout, et le chargement se comporte exactement comme décrit ici.

La porte de contrôle, avant d'aller plus loin : cinq modules d'étape, donc cinq fonctions.

```
findstr /R /C:"^def executer" pipeline\*.py
```

Piège — AttributeError: module 'pipeline.raw' has no attribute 'executer'. Les fichiers déplacés à l'étape 1 sont restés ceux des Cours 03 et 04 : ils portent main(), pas executer(con). La porte d'entrée va les chercher dès son dictionnaire ETAPES, c'est-à-dire à l'import : le pipeline meurt avant même d'ouvrir l'entrepôt, et --etapes n'y échappe pas. Reprenez le module fautif d'après l'annexe, puis videz pipeline__pycache__ si l'erreur tient toujours.

Vient enfin la porte d'entrée. Un paquet dont le dossier porte un fichier __main__.py se lance par python -m <paquet> : c'est ce fichier qui s'exécute. argparse, de la bibliothèque standard, lit la ligne de commande : il déclare les arguments attendus, les vérifie, et écrit lui-même le message d'aide.

```
notepad pipeline\__main__.py
```

```
# -*- coding: utf-8 -*-
```

```
"""__main__.py : La porte d'entrée du pipeline de Boréal.
```

```
python -m pipeline bronze raw corpus      Les étapes nommées, dans cet ordre
python -m pipeline tout                   Les étapes du chemin hors ligne, dans le bon ordre
python -m pipeline raw --bavard           Le journal descend au niveau DEBUG
python -m pipeline --etapes               La liste des étapes, sans rien lancer
```

```
"""
```

```
import argparse
```

```
import os
```

```
import sys
```

```
from . import bronze, corpus, erp, meteo, raw
```

```
from .config import projet
```

```
from .entrepot import ouvrir
```

```
from .journal import LOG, demarrer, etape
```

```
# Le nom de l'étape en ligne de commande, ce qu'elle fait, et si « tout » la prend.
```

```
ETAPES = {
```

```
    "bronze": (bronze.executer, "les CSV et le JSON de l'usine, rangés en Parquet", True),
```

```
    "raw": (raw.executer, "les 22 livraisons chargées dans la couche RAW", True),
```

```
    "corpus": (corpus.executer, "la documentation déposée telle quelle dans le bronze", True),
```

```
    "erp": (erp.executer, "les neuf tables de la base de l'ERP, par ODBC", False),
```

```
    "meteo": (meteo.executer, "les observations d'ECCC, par son API", False),
```

```
}
```

Le dictionnaire est la seule liste des étapes : ajouter une étape au pipeline, plus tard, sera une ligne ici. À la suite, la lecture de la ligne de commande et la résolution de ce qui a été demandé. Les deux se tiennent à l'écart du travail, ce qui les rend lisibles à voix haute et vérifiables sans entrepôt.

```
def lire_arguments(arguments=None) -> argparse.Namespace:
    analyseur = argparse.ArgumentParser(
        prog="python -m pipeline",
        description="Le pipeline de Véhicules Boréal inc. : une étape, plusieurs, ou tout.")
    analyseur.add_argument("etapes", nargs="*", metavar="ÉTAPE",
        help="les étapes à lancer, ou « tout »")
    analyseur.add_argument("--bavard", action="store_true", help="journal au niveau DEBUG")
    analyseur.add_argument("--etapes", dest="lister", action="store_true",
        help="liste les étapes et ce qu'elles font, sans rien lancer")
    return analyseur.parse_args(arguments)

def resoudre(demandees: list) -> list:
    """La liste des étapes à lancer, dans l'ordre. « tout » prend celles du chemin hors ligne."""
    if demandees == ["tout"]:
        return [nom for nom, (_, _, dans_tout) in ETAPES.items() if dans_tout]
    inconnues = [nom for nom in demandees if nom not in ETAPES]
    if inconnues:
        raise SystemExit(f"Étape inconnue : {' '.join(inconnues)}. "
            f"Connues : {' '.join(ETAPES)}, ou « tout ».")
    return demandees
```

Puis la fonction qui fait tourner tout cela, dans l'ordre : lire les options, se placer dans le projet, ouvrir le journal, ouvrir l'entrepôt une fois, lancer chaque étape sous sa trace, et fermer quoi qu'il arrive.

```
def main(arguments=None) -> int:
    options = lire_arguments(arguments)
    if options.lister or not options.etapes:
        print("Étapes du pipeline :")
        for nom, (_, quoi, dans_tout) in ETAPES.items():
            print(f" {nom:8} {quoi}{' ' if dans_tout else ' (hors de « tout »)'}")
        return 0

    a_lancer = resoudre(options.etapes)
    # Le pipeline travaille depuis Projet_Boreal, d'où qu'on l'appelle : Les chemins des scripts SQL
    # et le lignage de la RAW s'écrivent alors toujours de la même façon.
    os.chdir(projet())
    if options.bavard:
        os.environ["PIPELINE_JOURNAL_NIVEAU"] = "DEBUG"
    demarrer()
```

```

LOG.info("pipeline : %s, depuis %s", "", ".join(a_lancer), projet())

con = ouvrir()
try:
    for nom in a_lancer:
        action = ETAPES[nom][0]
        with etape(con, nom) as trace:
            # Une étape rend un entier, ce qu'elle a écrit. Le convertir ici fait dire à une
            # étape mal écrite ce qu'elle a fait de travers, plutôt qu'à DuckDB deux lignes
            # plus loin, dans une erreur de conversion de type.
            resultat = action(con)
            trace.lignes = None if resultat is None else int(resultat)
except Exception as erreur:
    LOG.error("pipeline arrêté : %s", erreur)
    return 1
finally:
    con.close()
LOG.info("pipeline : terminé")
return 0

if __name__ == "__main__":
    sys.stdout.reconfigure(encoding="utf-8", errors="replace")
    sys.exit(main())

```

Le pipeline est complet. Fermez la ligne de commande DuckDB si elle est ouverte, puis :

```

python -m pipeline --etapes
python -m pipeline tout

```

Attendu, sur une génération par défaut dont la RAW est déjà chargée par le Labo 03 : le bronze se refait (275 436 lignes d'historien et 158 146 de télémétrie, soit 433 582 en tout, en 6 à 7 secondes), la RAW ne charge rien et affiche « à jour » sur ses 22 lignes, et le corpus trouve ses 35 fichiers déjà déposés. Le pipeline se termine sur pipeline : terminé.

Pour voir un premier passage complet, effacez ce que le pipeline reconstruit, et cela seulement :

```

rmdir /s /q ..\plateforme\lac
rmdir /s /q ..\plateforme\entrepot
rmdir /s /q ..\plateforme\journaux
python -m pipeline tout

```

Trois dossiers, pas la plateforme entière. Ce qui reste debout ne vous appartient pas : le `docker-compose.yml`, le `Dockerfile.airflow` et le `.env` qui porte le mot de passe de la base sont arrivés tout prêts en tête du Cours 04, et aucun cours ne les recrée. Les effacer se paierait au Cours 06, quand Airflow n'aurait plus de pile à démarrer.

Piège — « tout » ne veut pas dire tout. La porte d'entrée refait le bronze, la RAW et le corpus, qui partent tous de `sources/`. Elle ne refait ni `erp`, qui a besoin du conteneur de la base, ni `meteo`, qui a besoin du réseau : le bronze `erp_base` et les observations de l'API restent donc en place, et c'est voulu. Si vous les effacez quand même, relancez `python -m pipeline erp` avec le conteneur démarré, puis `python -m pipeline meteo`.

Attendu cette fois, et ce sont les chiffres à retrouver :

Étape	Ce qu'elle écrit	Durée chez l'auteur
bronze	433 582 lignes (275 436 et 158 146)	6,6 s
raw	1 572 fichiers, 513 857 lignes, 22 tables	12,3 s
corpus	35 fichiers déposés	0,1 s

Relancez `python -m pipeline` tout une seconde fois : la RAW dit « à jour » partout et rend 0 ligne, le corpus trouve 35 fichiers déjà à jour. C'est l'idempotence du Cours 03, qui n'a pas bougé en changeant de forme.

Le code de sortie se vérifie aussi, et c'est lui que l'ordonnanceur du Cours 06 lira :

```
python -m pipeline erp
echo %ERRORLEVEL%
```

Attendu, tant que `psqlODBC` n'est pas installé : le journal montre étape `erp (...)` : échec, Pilote `psqlODBC` introuvable, et `%ERRORLEVEL%` vaut 1. Une étape qui échoue arrête le pipeline, l'inscrit dans la table, et le dit à qui l'a lancé.

Regardez enfin ce que la table a gardé. Dans la ligne de commande DuckDB, lancée depuis `Projet_Boreal` :

```
duckdb
```

La ligne de commande s'ouvre sur une base en mémoire, vide : l'invite affiche `memory D`. Il faut lui dire quel entrepôt lire, comme aux Cours 03 et 04. En lecture seule, puisqu'on ne fait que regarder :

```
ATTACH IF NOT EXISTS 'plateforme/entrepot/boreal.duckdb' AS entrepot (READ_ONLY);
USE entrepot;

-- L'histoire des passages, le plus récent en dernier.
-- Attendu : une ligne par étape et par passage, « réussi » partout sauf pour erp, « échoué ».
SELECT etape, etat, lignes, ROUND(duree_s, 1) AS duree_s, LEFT(message, 40) AS message
FROM pipeline.executions
ORDER BY debut;
```

Piège — Table with name "pipeline.executions" does not exist because schema "pipeline" does not exist. L'ATTACH a été oublié, ou la ligne de commande a été rouverte depuis. La table existe, mais dans le fichier, pas dans la base en mémoire où l'invite vous a déposé. L'invite le dit : memory D avant l'ATTACH, entrepot D après.

Quittez DuckDB par `.exit` avant l'étape suivante : un seul processus à la fois sur l'entrepôt.

Point d'arrêt. Le pipeline est écrit, il tourne, et ses passages sont dans la table. Si vous faites ce labo en deux séances, la première s'arrête ici : les étapes 6 à 8 ajoutent les tests, l'image et le commit, et ne supposent rien de plus que le dépôt tel qu'il est maintenant.

Étape 6 — Écrire trois tests

Théorie : Theorie_05.md, section 6.

Toujours dans l'Anaconda Prompt, depuis 01_de. Trois tests, un par promesse du pipeline, sur des données qu'ils fabriquent (théorie, section 6).

D'abord, dire à pytest où chercher :

```
notepad pytest.ini
```

```
# pytest.ini : où pytest cherche les tests, et où il cherche le paquet.  
#  
# Sans pythonpath, « pytest » lancé depuis 01_de s'arrête sur « ModuleNotFoundError: No module named  
# 'pipeline' », alors que « python -m pytest » passe : la seconde forme ajoute le dossier courant aux  
# chemins d'importation, la première non. Ce fichier fait que les deux marchent.
```

```
[pytest]  
pythonpath = .  
testpaths = tests
```

Trois outils de pytest servent dans le fichier qui suit. `tmp_path` donne à chaque test un dossier vide à lui, effacé ensuite. `monkeypatch` change une variable d'environnement ou un attribut le temps du test, puis remet tout en place. Et `cache_clear`, que `lru_cache` ajoute à la fonction qu'il décore, jette ce qu'elle avait garde : sans lui, le second test relirait la configuration du premier.

```
notepad tests\test_pipeline.py
```

```
# -*- coding: utf-8 -*-  
"""test_pipeline.py : trois propriétés du pipeline, vérifiées sans les données de Boréal."""  
import urllib.error  
  
import pytest  
  
from pipeline import bronze, raw  
from pipeline.config import config
```

```

from pipeline.entrepot import ouvrir
from pipeline.meteo import passagere
from pipeline.reprise import Abandon, avec_reprises

```

```

@pytest.fixture
def projet(tmp_path, monkeypatch):
    """Un Projet_Boreal jetable : Le pipeline y écrit son entrepôt et ses fichiers."""
    monkeypatch.setenv("PIPELINE_PROJET", str(tmp_path))
    config.cache_clear()
    monkeypatch.chdir(tmp_path)
    yield tmp_path
    config.cache_clear()

```

```

def ecrire_extrait(projet, mois: str, lignes: list) -> None:
    dossier = projet / "sources" / "erp" / "extraits" / mois
    dossier.mkdir(parents=True, exist_ok=True)
    contenu = "VBELN,NETWR\n" + "".join(f"{v}\n" for v in lignes)
    (dossier / "VBAK.csv").write_text(contenu, encoding="utf-8")

```

Le premier test, à la suite dans le même fichier. Il tient la promesse du Cours 03 : le journal des chargements reconnaît ce qu'il a déjà vu, remplace ce qui a changé, et laisse passer une livraison qui porte un autre nom.

```

def test_le_journal_refuse_un_extrait_deja_charge(projet):
    """Recharger Le même fichier ne double rien ; Le même fichier corrigé remplace ses lignes."""
    ecrire_extrait(projet, "2023-01", ["0000000001,100", "0000000002,200"])
    con = ouvrir()
    try:
        motif, table = "sources/erp/extraits/*/VBAK.csv", "erp_vbak"

        fichiers, lignes = raw.charger_une(con, table, motif, "csv", "dossier")
        assert (fichiers, lignes) == (1, 2)

        # Deuxième passage, rien n'a changé : Le journal reconnaît l'empreinte et ne recharge rien.
        assert raw.charger_une(con, table, motif, "csv", "dossier") == (0, 0)
        assert con.execute(f"SELECT COUNT(*) FROM raw.{table}").fetchone()[0] == 2

        # Le même nom, un autre contenu : ses anciennes lignes partent, Les nouvelles entrent.
        ecrire_extrait(projet, "2023-01", ["0000000001,100", "0000000002,200", "0000000003,300"])
        assert raw.charger_une(con, table, motif, "csv", "dossier") == (1, 3)
        assert con.execute(f"SELECT COUNT(*) FROM raw.{table}").fetchone()[0] == 3
    finally:
        con.close()

```

```

# Un autre nom de dossier est une autre livraison : Le journal La laisse passer, et la RAW
# garde les deux lots. C'est le piège de 2023-03_bis, et il est voulu.
ecrire_extrait(projet, "2023-01_bis", ["000000001,100"])
assert raw.charger_une(con, table, motif, "csv", "dossier") == (1, 1)
assert con.execute(f"SELECT COUNT(DISTINCT _lot) FROM raw.{table}").fetchone()[0] == 2
finally:
    con.close()

```

Le deuxième porte sur la conversion du Cours 02 : son contrôle doit compter pareil des deux côtés, et un script qui ne finit pas par un contrôle est refusé plutôt que cru sur parole.

```

def test_la_conversion_garde_le_compte_des_lignes(projet, tmp_path):
    """Le contrôle d'un script de conversion rend le même compte des deux côtés, ou il lève."""
    source = projet / "mesures"
    source.mkdir()
    for jour, valeurs in (("2025-10-01", [1.0, 2.0]), ("2025-10-02", [3.0]]):
        (source / f"mesures_{jour}.csv").write_text(
            "Tag,Value\n" + "".join(f"UTL.CMP2.VIBRATION,{v}\n" for v in valeurs), encoding="utf-8")

    script = tmp_path / "conversion.sql"
    script.write_text("""
COPY (
    SELECT Tag, Value,
           CAST(REPLACE(PARSE_FILENAME(filename, true), 'mesures_', '') AS DATE) AS jour
    FROM read_csv('mesures/*.csv', header = true, filename = true,
                  columns = {'Tag': 'VARCHAR', 'Value': 'DOUBLE'})
    ) TO 'bronze/mesures' (FORMAT parquet, PARTITION_BY (jour), OVERWRITE);
    SELECT (SELECT COUNT(*) FROM 'mesures/*.csv') AS lignes_csv,
           (SELECT COUNT(*) FROM 'bronze/mesures/*.parquet') AS lignes_parquet;
    """, encoding="utf-8")
    (projet / "bronze").mkdir()

    con = ouvrir()
    try:
        assert bronze.convertir(con, script) == (3, 3)

        # Un script qui ne finit pas par un contrôle à deux colonnes est refusé, plutôt que cru.
        muet = tmp_path / "muet.sql"
        muet.write_text("SELECT 1;", encoding="utf-8")
        with pytest.raises(bronze.EcartDeConversion):
            bronze.convertir(con, muet)
    finally:
        con.close()

```

Le troisième porte sur la reprise écrite à l'étape 4 : le nombre d'essais, les attentes qui doublent, et une erreur définitive qui ne se rejoue pas.

```
def test_la_reprise_s_arrete_apres_le_nombre_d_essais_prevu(projet):
    """Quatre essais, trois attentes qui doublent, puis l'abandon. Une erreur définitive ne passe pas."""
    essais = []
    attentes = []

    def toujours_en_panne():
        essais.append(1)
        raise urllib.error.HTTPError("https://exemple", 503, "Service Unavailable", None, None)

    with pytest.raises(Abandon):
        avec_reprises(toujours_en_panne, passagere, "essai", essais=4, attente=2.0,
                      dormir=attentes.append)
    assert len(essais) == 4
    assert attentes == [2.0, 4.0, 8.0]

    # Un 404 dit que la demande est fausse : une seule tentative, et l'erreur remonte telle quelle.
    definitive = []

    def demande_fausse():
        definitive.append(1)
        raise urllib.error.HTTPError("https://exemple", 404, "Not Found", None, None)

    with pytest.raises(urllib.error.HTTPError):
        avec_reprises(demande_fausse, passagere, "essai", essais=4, attente=2.0,
                      dormir=attentes.append)
    assert len(definitive) == 1
```

Le fichier s'arrête là. Ce qui suit se tape dans l'Anaconda Prompt, depuis 01_de, et non dans le fichier : pytest est le lanceur, pas du code de test. Il cherche les fonctions dont le nom commence par test_ dans tests/, les exécute une par une, et compte.

pytest

Attendu : 3 passed, en moins d'une seconde (0,27 s chez l'auteur). Les trois tests n'ont touché ni sources/, ni la plateforme, ni le réseau.

Le troisième test mérite un regard : il ne dort jamais. La fonction avec_reprises reçoit sa façon d'attendre en argument, et le test lui donne une liste qui note les délais au lieu de les subir. La vérification est alors plus forte que « ça a marché » : elle affirme que les attentes valent 2, 4 et 8 secondes, dans cet ordre.

Piège — le test qui passe parce qu'il ne teste rien. Retirez volontairement la ligne `if connus.get(f) != (sceau := empreinte(f))` de `raw.a_charger`, en la remplaçant par une condition toujours vraie, et relancez pytest. Le premier test doit échouer. S'il passe encore, c'est lui qu'il faut corriger, pas le code.

Étape 7 — Mettre le pipeline dans une image, et la lancer

Théorie : Theorie_05.md, section 8.

Deux terminaux à partir d'ici. On reste dans l'Anaconda Prompt, depuis 01_de, pour écrire les deux fichiers, puis on passe dans Ubuntu pour Docker.

Les deux fichiers vont à la racine de 01_de, à côté de pipeline\ et de pytest.ini, et non dans un sous-dossier : c'est ce dossier que docker build enverra au moteur.

notepad Dockerfile

Piège — Dockerfile n'a aucune extension. C'est le cas le plus exposé au .txt du Bloc-notes, vu à l'étape 1 : un nom sans point n'a rien qui ressemble à une extension, et l'enregistrement en ajoute une. Vérifiez avant d'aller plus loin, dans l'Anaconda Prompt : dir Dockerfile* doit montrer Dockerfile, et rien d'autre. S'il montre Dockerfile.txt, renommez par ren Dockerfile.txt Dockerfile. Le .dockerignore qui suit ne court pas le même risque, son point initial suffit au Bloc-notes.

```
# Dockerfile : Le pipeline de Boréal, empaqueté dans une image.  
#  
# L'image fige Python, DuckDB et Le code. Elle ne contient ni données ni secrets : Le projet arrive  
# par un montage, le mot de passe par une variable d'environnement.
```

```
FROM python:3.11-slim
```

```
# La version est fixée : une image qu'on reconstruit dans six mois donne le même moteur SQL.  
RUN pip install --no-cache-dir duckdb==1.5.5
```

```
WORKDIR /app/01_de  
COPY pipeline/ pipeline/  
COPY sql/ sql/  
COPY config.toml ./
```

```
# Le projet n'est pas dans l'image : il arrive par un montage, à cet endroit.  
ENV PIPELINE_PROJET=/travail  
ENV PYTHONUNBUFFERED=1
```

```
ENTRYPOINT ["python", "-m", "pipeline"]  
CMD ["--etapes"]
```

Les deux dernières lignes du fichier se lisent ensemble : ENTRYPOINT fixe la commande que le conteneur lance toujours, CMD donne les arguments par défaut, que la ligne de commande remplace. docker run <image> lance donc la porte d'entrée avec --etapes, et docker run <image> bronze la lance avec bronze.

notepad .dockerignore

```
# .dockerignore : ce que la construction de l'image ne doit même pas voir.
#
# Docker envoie le dossier courant au moteur avant de construire. Ce qui est listé ici ne part pas,
# donc ne peut pas se retrouver dans l'image par accident.

# Le secret ne voyage pas dans une image : il arrive par l'environnement, au lancement.
.env

# Les tests appartiennent à l'atelier, pas au pipeline livré.
tests/
pytest.ini

__pycache__/
.pytest_cache/
```

Passez maintenant dans Ubuntu (WSL 2). Le plus simple est celui du Labo 04 : depuis 01_de, dans l'Anaconda Prompt, tapez `wsl`, et Ubuntu s'ouvre dans le même dossier, sans avoir à retaper un chemin. L'invite l'affiche en `/mnt/c/.../01_de` : sous Ubuntu, le disque C de Windows se lit sous `/mnt/c`.

```
wsl
pwd
docker build -t boreal-pipeline .
```

`pwd` doit finir par `/Data_Domain/Projet_Boreal/01_de`. S'il montre autre chose, le `wsl` a été tapé depuis un autre dossier : ressortez par `exit`, replacez-vous dans 01_de, et recommencez.

Attendu : la construction se termine sur `naming to docker.io/library/boreal-pipeline:latest`. Elle prend une minute la première fois, quelques secondes ensuite.

Lancez le pipeline dans le conteneur, avec le projet monté sur `/travail` :

```
docker run --rm -v "$(cd .. && pwd)":/travail boreal-pipeline raw corpus
```

Attendu : les 22 livraisons « à jour », RAW : 0 fichier(s) chargé(s) à ce passage, et corpus : 0 fichier(s) déposé(s), 35 déjà à jour. Le conteneur a lu l'entrepôt écrit depuis Windows, reconnu chaque fichier par son empreinte, et inscrit ses propres lignes dans `pipeline.executions`.

Deux observations valent le détour.

La première : le lignage de la RAW est écrit en chemins relatifs, avec des barres obliques (`sources/erp/extraits/2023-01/VBAK.csv`). C'est pour cela que le conteneur, sous Linux, reconnaît ce que Windows a chargé. Un chemin absolu, ou des barres inversées, auraient fait recharger les 1 572 fichiers.

La seconde : l'étape `raw` prend 6,5 s dans le conteneur contre 0,1 s sous Windows, pour le même travail, celui de calculer 1 572 empreintes. La différence est le coût de la traversée : les fichiers de Windows vus depuis un conteneur Linux passent par une couche de traduction. C'est le coût annoncé par la décision D59 du parcours, et il se mesure ici pour la première fois.

Piège — le conteneur ne vit pas à la même heure que vous. Un conteneur Linux tourne en temps universel coordonné, UTC. Regardez `plateforme/journaux/` : à partir de 20 h heure de l’Est, le conteneur écrit dans le fichier du lendemain, pendant que Windows écrit dans celui du jour. Deux fichiers, une seule soirée de travail.

Étape 8 — Versionner le code, laisser dehors l’image et les données

Théorie : Theorie_05.md, section 3.

Retour dans l’Anaconda Prompt, depuis `01_de`.

```
git status
git add pipeline tests config.toml pytest.ini Dockerfile .dockerignore .gitignore
git status
git commit -m "Cours 05 : le pipeline en paquet, configure, journalise, teste et conteneurise"
```

Le second `git status` est le geste qui compte. Lisez la liste avant de valider, et vérifiez que `.env` n’y est pas : il est ignoré, mais l’ignorer n’est utile que si on le vérifie. Un mot de passe validé une fois reste dans l’historique, même retiré au commit suivant.

Récapitulatif

L’état du pipeline à la fin de ce labo. Dans la ligne de commande DuckDB, depuis `Projet_Boreal` :

```
-- Ce que le pipeline a produit, et ce que ses passages ont laissé comme trace.
-- Attendu : 22 tables dans raw, 513 857 lignes chargées, et au moins une ligne par étape.
SELECT
  (SELECT COUNT(*) FROM information_schema.tables WHERE table_schema = 'raw') AS tables_raw,
  (SELECT SUM(lignes) FROM pipeline.chargements) AS lignes_raw,
  (SELECT COUNT(*) FROM pipeline.executions) AS passages,
  (SELECT COUNT(*) FROM pipeline.executions WHERE etat = 'échoué') AS passages_echoues;
```

Ce que vous avez maintenant	Où
Le paquet pipeline, une porte d’entrée, neuf modules	<code>01_de/pipeline/</code>
La configuration, hors du code	<code>01_de/config.toml</code>
Le secret, hors du dépôt	<code>01_de/.env</code> , ignoré par Git
Les journaux de texte, un par jour	<code>plateforme/journaux/</code>
La trace interrogeable des passages	<code>pipeline.executions</code>

Ce que vous avez maintenant	Où
Trois tests, en moins d'une seconde	01_de/tests/, lancés par pytest
L'image du pipeline	boreal-pipeline, dans Docker

Le Cours 06 prend la suite : le DAG d'Airflow appellera cette porte d'entrée, tâche par tâche, et c'est la table des exécutions qui dira ce qu'il a vraiment fait.

Dépannage

Symptôme	Cause	Ce qu'on fait
failed to read dockerfile: open Dockerfile: no such file or directory, avec transferring dockerfile: 2B	le Bloc-notes a enregistré Dockerfile.txt : le nom n'a pas d'extension, il en a ajouté une	ren Dockerfile.txt Dockerfile dans l'Anaconda Prompt, ou mv Dockerfile.txt Dockerfile dans Ubuntu
SyntaxError sur une ligne comme pythonpath = . dans tests\test_pipeline.py	le contenu de pytest.ini a été collé dans le fichier de tests	retirer du fichier tout ce qui précède # -*- coding: utf-8 -*-
ModuleNotFoundError: No module named 'pipeline' avec pytest	le dossier 01_de n'est pas dans les chemins d'importation	vérifier que pytest.ini existe, avec pythonpath = . ; ou lancer python -m pytest
ModuleNotFoundError: No module named 'pipeline' avec python -m pipeline	vous n'êtes pas dans 01_de	cd vers Data_Domain\Projet_Boreal\01_de
L'entrepôt est occupé : fermez la ligne de commande DuckDB	un autre processus tient boreal.duckdb	.exit dans la ligne de commande DuckDB, et fermer un pipeline resté ouvert
FileNotFoundError: ...config.toml est introuvable	le fichier n'est pas à côté du dossier pipeline	il va dans 01_de/, pas dans 01_de/pipeline/
ERP_MOT_DE_PASSE n'est ni dans l'environnement ni dans ...\env	le .env du Labo 04 manque	copier .env.exemple en .env et le remplir
IO Error: Failed to create directory à l'étape bronze	les dossiers parents du COPY partitionné manquent	c'est le mkdir de bronze.executer : vérifier qu'il est bien avant convertir
AttributeError: module 'pipeline.raw' has no attribute 'executer'	le module est encore le script du Cours 03, avec main()	le reprendre d'après l'annexe, comme le demande l'étape 5, puis vider pipeline__pycache__

Symptôme	Cause	Ce qu'on fait
Conversion Error: Unimplemented type for cast à la fin d'une étape	l'étape rend autre chose qu'un entier	chaque <code>executer</code> rend un seul nombre, celui des lignes écrites
<code>git mv</code> répond <code>Permission denied</code> sans coupable visible	un verrou passager de Windows	fermer les éditeurs et terminaux qui regardent ces fichiers, puis réessayer
<code>docker: permission denied while trying to connect</code>	votre compte Ubuntu n'est pas dans le groupe <code>docker</code>	c'est l'installation de Docker, à l'étape 5 du guide d'installation : <code>sudo usermod -aG docker \$USER</code> , puis rouvrir Ubuntu
Le conteneur recharge les 1 572 fichiers	le lignage a été écrit en chemins absolus ou à barres inversées	vérifier que <code>raw.a_charger</code> liste les fichiers par <code>Path(".").glob(motif)</code> et <code>as_posix()</code>
<code>pytest</code> passe alors que vous avez cassé le code exprès	le test ne vérifie pas ce qu'il prétend	c'est le test qu'il faut corriger, pas le code

Annexe — Tout le code, d'un seul tenant

Le corps du labo montre en entier `__init__.py`, `config.py`, `journal.py`, `entrepot.py`, `reprise.py`, `bronze.py`, `__main__.py`, `config.toml`, `pytest.ini`, `tests/test_pipeline.py`, le `Dockerfile` et le `.dockerignore`. Restent les quatre modules repris des Cours 03 et 04, que l'étape 5 fait reprendre : les voici en entier, dans l'état où ce labo les laisse.

Module	Ce qui a changé depuis les Cours 03 et 04
<code>raw.py</code>	plus de <code>main</code> ni d'ouverture de l'entrepôt ; <code>executer(con)</code> rend les lignes chargées ; les <code>print</code> deviennent des lignes de journal ; <code>a_charger</code> est sortie de <code>charger_une</code> , ce qui la rend testable
<code>meteo.py</code>	la boucle d'essais part dans <code>reprise.py</code> , la station et les dates viennent de <code>config()</code> , l'extraction incomplète lève au lieu de sortir
<code>erp.py</code>	<code>pyodbc</code> importé dans la fonction, la chaîne de connexion bâtie depuis <code>config()</code> et <code>secret()</code>
<code>corpus.py</code>	les chemins viennent de <code>chemin()</code> , le compte part au journal

`pipeline\raw.py`, le chargeur de la couche RAW, et le plus long des quatre :

```
# -*- coding: utf-8 -*-
```

```
"""raw.py : Les 22 livraisons de Boréal chargées dans La couche RAW (Cours 03, empaqueté au Cours 05).
```

Ce qu'il fait. Pour chacune des livraisons de LIVRAISONS, il cherche Les fichiers dans sources/, calcule l'empreinte SHA-256 de chacun, et ne charge que ceux que Le journal des chargements ne connaît pas encore, ou connaît avec une autre empreinte. Chaque fichier entre tel quel dans sa table de L'entrepôt : toutes les colonnes en texte, un objet JSON dans une colonne JSON, et trois colonnes de lignage, _fichier, _lot et _charge_le.

La règle qu'il illustre. L'idempotence : relancer ne double rien, parce que Le journal se souvient de chaque fichier chargé. Un fichier livré de nouveau sous Le même nom, avec un autre contenu, remplace ses propres lignes : c'est ainsi qu'un export complet se met à jour. Un fichier livré sous un autre nom est une autre livraison : L'extrait 2023-03_bis passe, et La RAW Le garde, marqué par son lot.

Ce que Le Cours 05 a changé, par rapport au script du Cours 03. Le module ne se lance plus seul et n'ouvre plus L'entrepôt : il reçoit La connexion de La porte d'entrée, écrit dans Le journal plutôt qu'à L'écran, et rend ses comptes à L'appelant, qui Les inscrit dans pipeline.executions. Les chemins viennent de La configuration. La logique de chargement, elle, n'a pas bougé d'une ligne.

```
import hashlib
from datetime import datetime, timezone
from pathlib import Path
```

```
from .entrepot import deja_charge
from .fenetre import Fenetre
from .journal import LOG
```

```
# Une livraison : La table de La RAW, Le motif de ses fichiers, Leur forme, et d'où vient Le lot.
# dossier : Le dossier du mois (2023-03, 2023-03_bis) ; jour : La date du nom de fichier ;
# complet : un export livré en entier, qui remplace Le précédent.
```

```
LIVRAISONS = [
    ("erp_t005u", "sources/erp/extraits/*/T005U.csv", "csv", "dossier"),
    ("erp_kna1", "sources/erp/extraits/*/KNA1.csv", "csv", "dossier"),
    ("erp_mara", "sources/erp/extraits/*/MARA.csv", "csv", "dossier"),
    ("erp_makt", "sources/erp/extraits/*/MAKT.csv", "csv", "dossier"),
    ("erp_vbak", "sources/erp/extraits/*/VBAK.csv", "csv", "dossier"),
    ("erp_vbap", "sources/erp/extraits/*/VBAP.csv", "csv", "dossier"),
    ("erp_cdhdr", "sources/erp/extraits/*/CDHDR.csv", "csv", "dossier"),
    ("erp_cdpos", "sources/erp/extraits/*/CDPOS.csv", "csv", "dossier"),
    ("erp_zserie", "sources/erp/extraits/*/ZSERIE.csv", "csv", "dossier"),
    ("garanties_clients", "sources/garanties/*/clients.csv", "csv", "dossier"),
    ("garanties_enregistrements", "sources/garanties/*/enregistrements.csv", "csv", "dossier"),
```

```

("garanties_visites", "sources/garanties/*/visites.csv", "csv", "dossier"),
("garanties_consentements", "sources/garanties/*/consentements.csv", "csv", "dossier"),
("garanties_reclamations", "sources/garanties/*/reclamations.json", "json", "dossier"),
("usine_actifs", "sources/usine/referentiel/actifs.csv", "csv", "complet"),
("usine_etiquettes", "sources/usine/referentiel/etiquettes.csv", "csv", "complet"),
("usine_bons_travail", "sources/usine/gmao/bons_travail.csv", "csv", "complet"),
("meteo_stations", "sources/meteo/stations.csv", "csv", "complet"),
("meteo_quotidien", "sources/meteo/quotidien.csv", "csv", "complet"),
("meteo_horaire", "sources/meteo/horaire_7060400.csv", "csv", "complet"),
("telemetrie_sorties", "sources/telemetrie/sorties_*.jsonl", "jsonl", "jour"),
("corpus_index", "sources/corpus/index.csv", "csv", "complet"),
]

# La requête qui lit une liste de fichiers, selon leur forme. Chacune rend la colonne filename.
LECTEURS = {
    "csv": "SELECT * FROM read_csv($fichiers, header = true, all_varchar = true, filename = true)",
    "json": ("SELECT json AS objet, filename "
            "FROM read_json_objects($fichiers, format = 'array', filename = true)"),
    "jsonl": ("SELECT sujet, recu_le, qos, message, filename "
            "FROM read_json($fichiers, format = 'newline_delimited', filename = true, "
            "columns = {'sujet': 'VARCHAR', 'recu_le': 'VARCHAR', 'qos': 'VARCHAR', 'message': 'JSON'})"),
}

def lot_du(fichier: str, regle: str) -> str:
    """Le lot d'un fichier : son dossier, le jour de son nom, ou « complet »."""
    chemin = Path(fichier)
    if regle == "dossier":
        return chemin.parent.name
    if regle == "jour":
        return chemin.stem.replace("sorties_", "")
    return "complet"

def empreinte(fichier: str) -> str:
    """L'empreinte SHA-256 du contenu : deux fichiers identiques la partagent, un octet la change."""
    return hashlib.sha256(Path(fichier).read_bytes()).hexdigest()

def a_charger(con, table: str, motif: str, regle: str, fenetre: Fenetre = None) -> list:
    """Les fichiers du motif que le journal ne connaît pas, ou connaît sous une autre empreinte.
```

Les chemins se lisent depuis Projet_Boreal, où la porte d'entrée s'est placée : Le lignage de la RAW s'écrit alors de la même façon sous Windows et sous Linux.

Deux filtres, dans cet ordre, et l'ordre compte (Cours 06). La fenêtre écarte d'abord les fichiers dont le lot tombe hors de la tranche de cette exécution : elle ne lit rien, elle regarde des noms. Le journal écarte ensuite ceux qu'il a déjà chargés sous la même empreinte : lui doit lire chaque fichier restant pour en calculer l'empreinte. Filtrer par la fenêtre d'abord, c'est ne pas relire 1 572 fichiers pour en charger 14.

```
fenetre = fenetre or Fenetre()
connus = deja_charge(con, table)
fichiers = [p.as_posix() for p in Path(".").glob(motif)]
dans_la_tranche = sorted(f for f in fichiers if fenetre.contient(lot_du(f, regle)))
return [(f, sceau, lot_du(f, regle)) for f in dans_la_tranche
        if connus.get(f) != (sceau := empreinte(f))]
```

```
class LivraisonInconnue(ValueError):
    """Un préfixe de livraison qui n'existe pas, passé par --Livraisons."""
```

```
def charger_une(con, table: str, motif: str, forme: str, regle: str,
               fenetre: Fenetre = None) -> tuple:
    """Charge les fichiers nouveaux ou modifiés d'une livraison. Rend (fichiers, lignes)."""
    nouveaux = a_charger(con, table, motif, regle, fenetre)
    if not nouveaux:
        return 0, 0

    source = LECTEURS[forme]
    noms = [f for f, _, _ in nouveaux]
    charge_le = datetime.now(timezone.utc)
    con.execute("BEGIN")
    try:
        # La table naît de la forme du premier fichier : ses colonnes, en texte, puis le lignage.
        con.execute(f"""
            CREATE TABLE IF NOT EXISTS raw.{table} AS
            SELECT * EXCLUDE (filename), filename AS _fichier, '' AS _lot, NOW() AS _charge_le
            FROM ({source}) LIMIT 0""", {"fichiers": noms[:1]})
        con.execute("CREATE OR REPLACE TEMP TABLE a_charger "
                   "(fichier VARCHAR, empreinte VARCHAR, lot VARCHAR)")
        con.executemany("INSERT INTO a_charger VALUES (?, ?, ?)", nouveaux)
        # Un fichier déjà chargé sous une autre empreinte : ses anciennes lignes partent d'abord.
```

```

con.execute(f"DELETE FROM raw.{table} WHERE _fichier IN (SELECT fichier FROM a_charger)")
con.execute(f"""
    INSERT INTO raw.{table}
    SELECT s.* EXCLUDE (filename), s.filename, a.lot, $charge_le
    FROM ({source}) s JOIN a_charger a ON a.fichier = s.filename""",
        {"fichiers": noms, "charge_le": charge_le})
con.execute(f"""
    INSERT INTO pipeline.chargements
    SELECT a.fichier, a.empreinte, '{table}', a.lot, COUNT(r._fichier), $charge_le
    FROM a_charger a LEFT JOIN raw.{table} r
        ON r._fichier = a.fichier AND r._charge_le = $charge_le
    GROUP BY ALL""", {"charge_le": charge_le})
lignes = con.execute(f"SELECT COUNT(*) FROM raw.{table} WHERE _charge_le = ?",
                    [charge_le]).fetchone()[0]
con.execute("COMMIT")
except Exception:
    # Qui ouvre la transaction la referme. Sans ce ROLLBACK, DuckDB la garde avortée, et la
    # requête suivante, y compris celle qui ferme la ligne de l'étape, répond « Current
    # transaction is aborted » au lieu de laisser passer la vraie cause.
    con.execute("ROLLBACK")
    raise
LOG.debug("%s : %d fichier(s), %d lignes", table, len(noms), lignes)
return len(noms), lignes

```

```

def sources_de(livraisons=None) -> list:

```

"""Les livraisons demandées, par leur préfixe : « erp », « telemetrie »... Toutes par défaut.

Chaque DAG du Cours 06 nomme ainsi ce qu'il possède : Le mensuel prend l'ERP et les garanties, le quotidien prend l'usine, la télémétrie, la météo et le corpus. Deux DAG qui se partagent les livraisons ne se marchent pas dessus, et chacun se lit sans deviner ce que l'autre fait.

```

if not livraisons:
    return LIVRAISONS
connus = sorted({table.split("_")[0] for table, _, _, _ in LIVRAISONS})
inconnus = [nom for nom in livraisons if nom not in connus]
if inconnus:
    # Une erreur du domaine, jamais un SystemExit : celui-ci n'est pas une Exception, il
    # traverserait le gestionnaire d'étape sans fermer sa ligne ni passer par le journal.
    raise LivraisonInconnue(f"Livraison inconnue : {' '.join(inconnus)}. "
                            f"Connues : {' '.join(connus)}.")
return [ligne for ligne in LIVRAISONS if ligne[0].split("_")[0] in livraisons]

```

```
def executer(con, fenetre: Fenetre = None, livraisons=None) -> int:
    """Charge Les livraisons demandées. Rend Le nombre de Lignes chargées à ce passage.

    Comme toute étape du pipeline, elle rend un entier : c'est ce que la porte d'entrée inscrit dans
    la colonne lignes de pipeline.executions. Le compte des fichiers, lui, va au journal de texte.

    Sans fenêtre ni liste de livraisons, elle se comporte exactement comme au Cours 05 : Les 22
    livraisons, et tout ce que le journal ignore.
    """
    fenetre = fenetre or Fenetre()
    total_fichiers = total_lignes = 0
    for table, motif, forme, regle in sources_de(livraisons):
        fichiers, lignes = charger_une(con, table, motif, forme, regle, fenetre)
        if fichiers:
            LOG.info(" %-26s %4d fichier(s), %8s lignes", table, fichiers,
                    f"{lignes:,}".replace(",", " "))
        else:
            LOG.info(" %-26s à jour", table)
        total_fichiers += fichiers
        total_lignes += lignes
    LOG.info(" RAW : %s fichier(s) chargé(s) à ce passage", f"{total_fichiers:,}".replace(",", " "))
    return total_lignes
```

pipeline\meteo.py, les observations d'ECCC, dont l'étape 4 n'a montré que la lecture d'une page :

```
# -*- coding: utf-8 -*-
"""meteo.py : La vraie API d'ECCC, page par page, avec les reprises du pipeline (Cours 05).
```

Ce qu'il fait. Il demande à api.weather.gc.ca les observations quotidiennes de la station configurée, page par page, en suivant le lien « next » que chaque réponse fournit, et écrit chaque observation telle que reçue, une ligne JSON par jour, dans `plateforme/lac/bronze/meteo_api/`. À la fin, il compare les jours reçus aux jours annoncés par l'API et refuse une extraction incomplète.

Ce que le Cours 05 a changé, par rapport au script du Cours 04. La boucle d'essais du script est partie dans `pipeline.reprise`, où elle sert à toute source qui peut faiblir : le module ne garde que ce qui lui est propre, savoir quelles erreurs HTTP valent un second essai. La station, les dates et la taille de page viennent de la configuration, et les messages vont au journal.

La règle qu'il illustre. Ce qui se répète dans deux étapes n'appartient à aucune des deux. Une reprise, une journalisation, une lecture de configuration sont l'affaire du pipeline ; l'étape ne garde que la connaissance de sa source.

```

"""
import json
import urllib.error
import urllib.parse
import urllib.request

from .config import config, chemin
from .journal import LOG
from .reprise import avec_reprises

class ExtractionIncomplete(RuntimeError):
    """L'API a annoncé plus de jours qu'elle n'en a livré."""

def passagere(erreur: Exception) -> bool:
    """Un 429 ou un 5xx passeront peut-être ; un 4xx dit que la demande est fausse."""
    if isinstance(erreur, urllib.error.HTTPError):
        return erreur.code == 429 or erreur.code >= 500
    return isinstance(erreur, urllib.error.URLError) # coupure, DNS, délai dépassé

def lire_page(url: str) -> dict:
    """Une page de l'API, réessayée tant que l'erreur est passagère."""
    def demander():
        with urllib.request.urlopen(url, timeout=60) as reponse:
            return json.load(reponse)
    return avec_reprises(demander, passagere, "une page de l'API météo")

def executer(con=None, fenetre=None, livraisons=None) -> int:
    """Tire les observations de la station configurée. Rend le nombre de jours reçus.

    La fenêtre du Cours 06 ne s'applique pas ici : l'API porte ses propres dates, dans config.toml.
    Les brancher sur la fenêtre est la question pratique 6 du Cours 06.
    """
    reglages = config()["meteo"]
    parametres = {"f": "json", "CLIMATE_IDENTIFIER": reglages["station"], "sortby": "LOCAL_DATE",
                  "datetime": f"{reglages['debut']} 00:00:00/{reglages['fin']} 00:00:00",
                  "limit": reglages["page"]}
    url = f"{reglages['url']}?{urllib.parse.urlencode(parametres)}"
    sortie = (chemin("lac") / "bronze" / "meteo_api" /

```

```

        f"quotidien_{reglages['station']}_{reglages['debut']}_{reglages['fin']}.jsonl")
sortie.parent.mkdir(parents=True, exist_ok=True)

pages = recus = 0
annonces = None
with open(sortie, "w", encoding="utf-8", newline="\n") as fichier:
    while url:
        page = lire_page(url)
        pages += 1
        annonces = page["numberMatched"]
        for entite in page["features"]:
            fichier.write(json.dumps(entite["properties"], ensure_ascii=False) + "\n")
        recus += page["numberReturned"]
        LOG.debug(" page %d : %d jours", pages, page["numberReturned"])
        # La page suivante, si l'API en annonce une.
        url = next((lien["href"] for lien in page["links"] if lien["rel"] == "next"), None)

LOG.info(" météo : %d jours reçus sur %d annoncés, en %d pages", recus, annonces, pages)
if recus != annonces:
    raise ExtractionIncomplete(
        f"{recus} jours reçus pour {annonces} annoncés : l'extraction est incomplète.")
return recus

```

pipeline\erp.py, les neuf tables de la base, par ODBC :

```

# -*- coding: utf-8 -*-
"""erp.py : Les neuf tables de la base de L'ERP, lues par ODBC (Cours 04, empaqueté au Cours 05).

```

Ce qu'il fait. Il se connecte à la base de L'ERP de Boréal, dans son conteneur PostgreSQL, par le pilote psycopg2, lit chaque table du schéma sap par paquets, l'écrit dans un CSV temporaire, puis DuckDB la range en Parquet dans plateforme/lac/bronze/erp_base/.

La règle qu'il illustre. Une dépendance qui ne sert qu'à une étape s'importe dans cette étape, pas en tête du paquet. pyodbc et le pilote psycopg2 ne s'installent que pour cette extraction : le reste du pipeline, celui que rejoue le kit 02, n'importe que DuckDB et la bibliothèque standard, et tourne même quand pyodbc n'est pas là. Le mot de passe, lui, n'est ni dans le code ni dans config.toml : il est lu par config.secret, dans 01_de/.env ou dans l'environnement du conteneur.

Lancement : `python -m pipeline erp`, le conteneur de la base démarré.

```

import csv
import time

```

```

import duckdb

from .config import config, chemin, secret
from .journal import LOG

class PiloteAbsent(RuntimeError):
    """Aucun pilote ODBC de PostgreSQL n'est installé sur ce poste."""

def chaine_de_connexion(pyodbc) -> str:
    """La chaîne ODBC, pilote compris. Changer de base ne change que cette chaîne."""
    pilotes = [p for p in pyodbc.drivers() if p.startswith("PostgreSQL Unicode")]
    if not pilotes:
        raise PiloteAbsent(f"Pilote psqlODBC introuvable. Pilotes ODBC installés : {pyodbc.drivers()}")
    reglages = config()["erp"]
    return (f"DRIVER={{pilotes[0]}};SERVER={reglages['serveur']};PORT={reglages['port']};"
            f"DATABASE={reglages['base']};UID={reglages['utilisateur']};"
            f"PWD={secret('ERP_MOT_DE_PASSE')}")

def executer(con=None, fenetre=None, livraisons=None) -> int:
    """Extrait Les neuf tables de la base. Rend le total des lignes lues.

    La fenêtre du Cours 06 ne s'applique pas ici : L'ERP livre une base, pas des fichiers datés, et
    c'est justement ce que Le Cours 04 oppose aux extraits mensuels.
    """

    import pyodbc # dépendance de cette étape seulement

    reglages = config()["erp"]
    sortie = chemin("lac") / "bronze" / "erp_base"
    sortie.mkdir(parents=True, exist_ok=True)

    debut = time.perf_counter()
    total = 0
    connexion = pyodbc.connect(chaine_de_connexion(pyodbc))
    curseur = connexion.cursor()
    duck = duckdb.connect() # une base en mémoire : L'entrepôt n'est pas touché
    for table in reglages["tables"]:
        curseur.execute(f"SELECT * FROM sap.{table}")
        colonnes = [c[0].upper() for c in curseur.description]
        temporaire = sortie / f"{table}.csv"

```

```

lignes = 0
with open(temporaire, "w", newline="", encoding="utf-8") as fichier:
    ecrivain = csv.writer(fichier)
    ecrivain.writerow(colonnes)
    while paquet := curseur.fetchmany(reglages["paquet"]):
        ecrivain.writerows(paquet)
        lignes += len(paquet)
# Le bronze garde Le texte reçu ; DuckDB Le range en Parquet compressé.
duck.execute(f"""
    COPY (SELECT * FROM read_csv('{temporaire.as_posix()}', header = true, all_varchar = true))
    TO '{(sortie / f"{table}.parquet").as_posix()}' (FORMAT parquet, COMPRESSION zstd)""")
temporaire.unlink()
LOG.info("  sap.-%8s %8s lignes", table, f"{lignes:,}".replace(",", " "))
total += lignes
connexion.close()
LOG.info("  neuf tables extraites en %.1f s", time.perf_counter() - debut)
return total

```

pipeline\corpus.py, la documentation déposée telle quelle :

```

# -*- coding: utf-8 -*-
"""corpus.py : La documentation de Boréal déposée telle quelle dans Le bronze (Cours 03, empaqueté au 05).

```

Ce qu'il fait. Il copie chaque fichier de sources/corpus/ dans plateforme/lac/bronze/corpus/, en gardant l'arborescence, et saute ceux dont l'empreinte n'a pas bougé. Le catalogue, lui, entre dans la RAW comme une table, par l'étape raw.

La règle qu'il illustre. Tout ne se met pas en table. Un document se dépose tel quel, avec sa date et son empreinte, et c'est ce dépôt que la formation AI ira lire. Déposer reste une étape du pipeline : elle se journalise, se relance sans dégât, et rend un compte.

```

import hashlib
import shutil

```

```

from .config import chemin
from .journal import LOG

```

```

def empreinte(fichier) -> str:
    return hashlib.sha256(fichier.read_bytes()).hexdigest()

```

```

def executer(con=None, fenetre=None, livraisons=None) -> int:

```

""Dépose Le corpus dans le bronze. Rend le nombre de fichiers déposés à ce passage.

La fenêtre du Cours 06 ne s'applique pas ici : un document n'a pas de lot, et c'est son empreinte qui dit s'il a bougé.

""

```
source = chemin("sources") / "corpus"
bronze = chemin("lac") / "bronze" / "corpus"
deposes = identiques = 0
for fichier in sorted(p for p in source.rglob("*") if p.is_file()):
    cible = bronze / fichier.relative_to(source)
    if cible.exists() and empreinte(cible) == empreinte(fichier):
        identiques += 1
        continue
    cible.parent.mkdir(parents=True, exist_ok=True)
    # copyfile, et non copy2 : recopier les métadonnées échoue quand un conteneur Linux écrit
    # sur un montage de fichiers Windows, « [Errno 1] Operation not permitted », et l'étape n'en
    # a pas besoin. C'est l'empreinte du contenu qui dit si un document a bougé, pas sa date.
    # Trouvé au Cours 06, la première fois qu'Airflow a lancé cette étape depuis un conteneur.
    shutil.copyfile(fichier, cible)
    depose += 1
LOG.info(" corpus : %d fichier(s) déposé(s), %d déjà à jour", depose, identiques)
return depose
```

Ces quatre fichiers sont ceux du Solution/, au caractère près. Pour comparer les vôtres à la référence, sans les écraser :

```
git diff --no-index pipeline\raw.py ..\..\01_Data_Engineering\Cours\Solution\01_de\pipeline\raw.py
```